

DATA STRUCTURES AND ALGORITHMS

LECTURE 4

Lect. PhD. Oneț-Marian Zsuzsanna

Babeș - Bolyai University
Computer Science and Mathematics Faculty

2025 - 2026

- Containers
 - ADT Bag
 - ADT SortedBag
 - ADT Set
 - ADT SortedSet
 - ADT Matrix
 - ADT Map
 - ADT SortedMap
 - ADT MultiMap
 - ADT SortedMultimap
 - ADT Stack

- Containers II
 - ADT Queue
 - ADT Priority Queue
 - ADT Dequeue
 - ADT List
- Linked lists
 - Singly linked list

- There are many different containers, based on different properties:
 - do the elements have to be unique?
 - do the elements have positions assigned?
 - can we access any element or just some specific ones?
 - do we have simple elements, or key-value pairs?



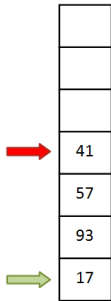
<http://www.rgbstock.com/photomeZ8AhAQueue+Line>

- Look at above queue of people waiting in front of a desk.
- If a new person arrives, where should he/she stand?
- When the blue person finishes, who is going to be the next at the desk?

- The ADT Queue represents a container in which access to the elements is restricted to the two ends of the container, called *front* and *rear*.
 - When a new element is added (pushed), it has to be added to the *rear* of the queue.
 - When an element is removed (popped), it will be the one at the *front* of the queue.
- Because of this restricted access, the queue is said to have a **FIFO** policy: First In First Out.

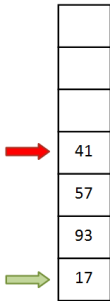
ADT Queue - Example

- Assume that we have the following queue (green arrow is the front, red arrow is the rear)
- Push number 33:

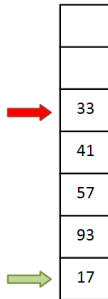


ADT Queue - Example

- Assume that we have the following queue (green arrow is the front, red arrow is the rear)



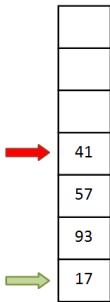
- Push number 33:



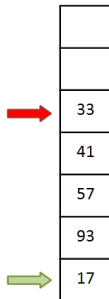
- Pop an element:

ADT Queue - Example

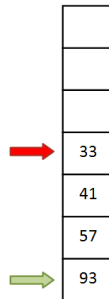
- Assume that we have the following queue (green arrow is the front, red arrow is the rear)



- Push number 33:

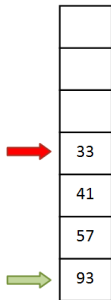


- Pop an element:



ADT Queue - Example

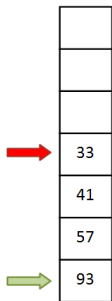
● This is our queue:



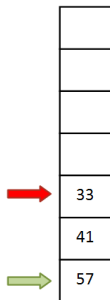
● Pop an element:

ADT Queue - Example

● This is our queue:



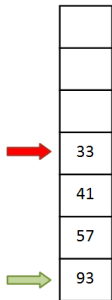
● Pop an element:



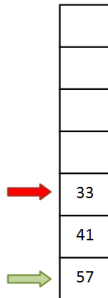
● Push number 72:

ADT Queue - Example

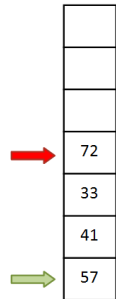
- This is our queue:



- Pop an element:



- Push number 72:



ADT Queue - Interface I

- The domain of the ADT Queue:
 $\mathcal{Q} = \{q \mid q \text{ is a queue with elements of type } TElem\}$
- The interface of the ADT Queue contains the following operations:

ADT Queue - Interface II

- **init(q)**
 - **descr:** creates a new empty queue
 - **pre:** True
 - **post:** $q \in \mathcal{Q}$, q is an empty queue

ADT Queue - Interface III

- **destroy(q)**
 - **descr:** destroys a queue
 - **pre:** $q \in Q$
 - **post:** q was destroyed

- **push**(q, e)
 - **descr:** pushes (adds) a new element to the rear of the queue
 - **pre:** $q \in \mathcal{Q}$, e is a *TElem*
 - **post:** $q' \in \mathcal{Q}$, $q' = q \oplus e$, e is the element at the rear of the queue

- **pop(q)**
 - **descr:** pops (removes) the element from the front of the queue
 - **pre:** $q \in \mathcal{Q}$, q is not empty
 - **post:** $pop \leftarrow e$, e is a *TElem*, e is the element at the front of q , $q' \in \mathcal{Q}$, $q' = q \ominus e$
 - **throws:** an *underflow* exception if the queue is empty

- **top(q)**
 - **descr:** returns the element from the front of the queue (but it does not change the queue)
 - **pre:** $q \in \mathcal{Q}$, q is not empty
 - **post:** $top \leftarrow e$, e is a $TElem$, e is the element from the front of q
 - **throws:** an *underflow* exception if the queue is empty

ADT Queue - Interface VII

- **isEmpty(s)**
 - **descr:** checks if the queue is empty (has no elements)
 - **pre:** $q \in \mathcal{Q}$
 - **post:**

$$isEmpty \leftarrow \begin{cases} \text{true, if } q \text{ has no elements} \\ \text{false, otherwise} \end{cases}$$

- **Note:** queues cannot be iterated, so they do not have an *iterator* operation!

ADT Queue - Representation

- What data structures can be used to implement a Queue?
 - Static Array - for a fixed capacity Queue
 - In this case an *isFull* operation can be added, and *push* can also throw an exception if the Queue is full.
 - Dynamic Array
 - other data structures (will be discussed later)

ADT Queue - Array-based representation

- If we want to implement a Queue using an array (static or dynamic), where should we place the *front* and the *rear* of the queue?

ADT Queue - Array-based representation

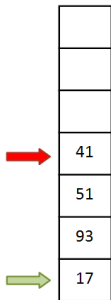
- If we want to implement a Queue using an array (static or dynamic), where should we place the *front* and the *rear* of the queue?
- In theory, we have two options:
 - Put *front* at the beginning of the array and *rear* at the end
 - Put *front* at the end of the array and *rear* at the beginning
- In either case we will have one operation (push or pop) that will have $\Theta(n)$ complexity.

ADT Queue - Array-based representation

- We can improve the complexity of the operations, if we do not insist on having either *front* or *rear* at the beginning of the array (at position 1).

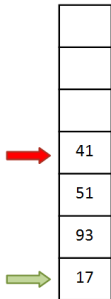
ADT Queue - Array-based representation

- This is our queue
(green arrow is the front, red arrow is the rear)
- Push number 33:

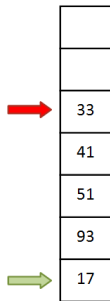


ADT Queue - Array-based representation

- This is our queue
(green arrow is the front, red arrow is the rear)



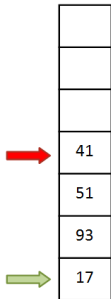
- Push number 33:



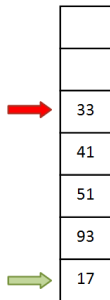
- Pop an element
(and do not move the other elements):

ADT Queue - Array-based representation

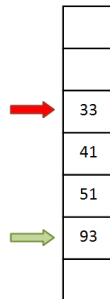
- This is our queue
(green arrow is the front, red arrow is the rear)



- Push number 33:



- Pop an element
(and do not move the other elements):

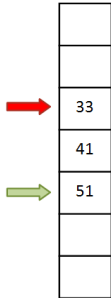


ADT Queue - Array-based representation

- Pop another element:

ADT Queue - Array-based representation

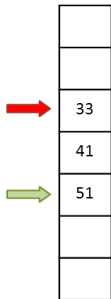
- Pop another element:



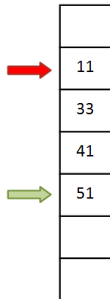
- Push number 11:

ADT Queue - Array-based representation

- Pop another element:



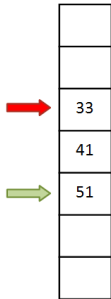
- Push number 11:



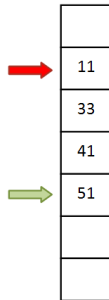
- Pop an element:

ADT Queue - Array-based representation

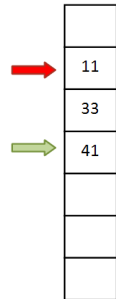
- Pop another element:



- Push number 11:



- Pop an element:

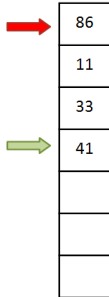


ADT Queue - Array-based representation

- Push number 86:

ADT Queue - Array-based representation

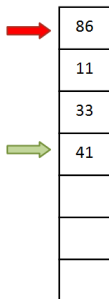
- Push number 86:



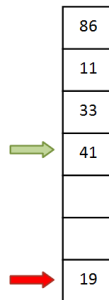
- Push number 19:

ADT Queue - Array-based representation

- Push number 86:



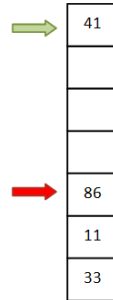
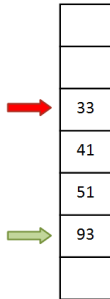
- Push number 19:



- This is called a **circular array**

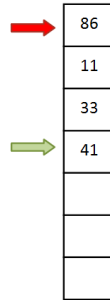
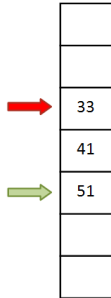
ADT Queue - representation on a circular array - pop

- There are two situations for our queue (green arrow is the front where we pop from):



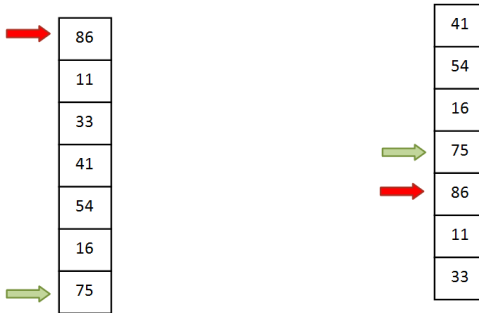
ADT Queue - representation on a circular array - push

- There are two situations for our queue (red arrow is the end where we push):



Queue - representation on a circular array - push

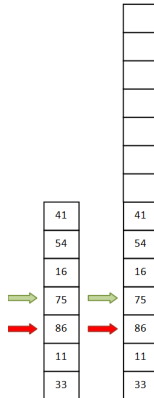
- When pushing a new element we have to check whether the queue is full



- For both example, the elements were added in the order: 75, 16, 54, 41, 33, 11, 86

ADT Queue - representation on a circular array - push

- If we have a dynamic array-based representation and the array is full, we have to allocate a larger array and copy the existing elements (as we always do with dynamic arrays)
- But we have to be careful how we copy the elements in order to avoid having something like:





Source: <https://www.vectorstock.com/royalty-free-vector/patients-in-doctors-waiting-room-at-the-hospital-vector-12041494>

- Consider the following queue in front of the Emergency Room. Who should be the next person checked by the doctor?

ADT Priority Queue

- The ADT Priority Queue is a container in which each element has an associated *priority* (of type *TPriority*).
- In a Priority Queue access to the elements is restricted: we can access only the element with the highest priority.
- Because of this restricted access, we say that the Priority Queue works based on a **HPF - Highest Priority First** policy.

ADT Priority Queue

- In order to work in a more general manner, we can define a relation $\mathcal{R}$ on the set of priorities: $\mathcal{R} : \text{TPriority} \times \text{TPriority}$
- When we say *the element with the highest priority* we will mean that the highest priority is determined using this relation $\mathcal{R}$.
- If the relation $\mathcal{R} = "\geq"$, the element with the *highest priority* is the one for which the value of the priority is the largest (maximum).
- Similarly, if the relation $\mathcal{R} = "\leq"$, the element with the *highest priority* is the one for which the value of the priority is the lowest (minimum).

Priority Queue - Interface I

- The domain of the ADT Priority Queue:
 $\mathcal{PQ} = \{pq \mid pq \text{ is a priority queue with elements } (e, p), e \in TElem, p \in TPriority\}$
- The interface of the ADT Priority Queue contains the following operations:

Priority Queue - Interface II

- **init** (pq, R)
 - **descr:** creates a new empty priority queue
 - **pre:** R is a relation over the priorities,
 $R : \mathcal{TPriority} \times \mathcal{TPriority}$
 - **post:** $pq \in \mathcal{PQ}$, pq is an empty priority queue

Priority Queue - Interface III

- **destroy**(pq)
 - **descr:** destroys a priority queue
 - **pre:** $pq \in \mathcal{PQ}$
 - **post:** pq was destroyed

Priority Queue - Interface IV

- **push**(pq, e, p)
 - **descr:** pushes (adds) a new element to the priority queue
 - **pre:** $pq \in \mathcal{PQ}, e \in TElem, p \in TPriority$
 - **post:** $pq' \in \mathcal{PQ}, pq' = pq \oplus (e, p)$

Priority Queue - Interface V

- **pop** (pq)
 - **descr:** pops (removes) from the priority queue the element with the highest priority. It returns both the element and its priority
 - **pre:** $pq \in \mathcal{PQ}$, pq is not empty
 - **post:** $pop \leftarrow (e, p)$, $e \in TElem$, $p \in TPriority$, e is the element with the highest priority from pq , p is its priority.
 $pq' \in \mathcal{PQ}$, $pq' = pq \ominus (e, p)$
 - **throws:** an exception if the priority queue is empty.

Priority Queue - Interface VI

- **top** (pq)
 - **descr:** returns from the priority queue the element with the highest priority and its priority. It does not modify the priority queue.
 - **pre:** $pq \in \mathcal{PQ}$, pq is not empty
 - **post:** $top \leftarrow (e, p)$, $e \in TElem$, $p \in TPriority$, e is the element with the highest priority from pq , p is its priority.
 - **throws:** an exception if the priority queue is empty.

- $\text{isEmpty}(pq)$
 - **Description:** checks if the priority queue is empty (it has no elements)
 - **Pre:** $pq \in \mathcal{PQ}$
 - **Post:**

$$\text{isEmpty} \leftarrow \begin{cases} \text{true, if } pq \text{ has no elements} \\ \text{false, otherwise} \end{cases}$$

Priority Queue - Interface VIII

- **Note:** priority queues cannot be iterated, so they don't have an *iterator* operation!

- The ADT Deque (Double Ended Queue) is a container in which we can insert and delete from both ends:
 - We have *push_front* and *push_back*
 - We have *pop_front* and *pop_back*
 - We have *top_front* and *top_back*
 - And obviously, *init* and *isEmpty*.
- **Note:** Specifications for these operations are similar to the specifications of the corresponding operations for the *stack* and *queue*
- We can simulate both stacks and queues with a deque if we restrict ourselves to using only part of the operations.

- A *list* can be seen as a sequence of elements of the same type, $\langle l_1, l_2, \dots, l_n \rangle$, where there is an order of the elements, and each element has a *position* inside the list.
- In a list, the order of the elements is important (positions are important).
- The number of elements from a list is called the length of the list. A list without elements is called *empty*.

- A List is a container which is either *empty* or
 - it has a unique *first* element
 - it has a unique *last* element
 - for every element (except for the last) there is a unique *successor* element
 - for every element (except for the first) there is a unique *predecessor* element
- In a list, we can insert elements (using positions), remove elements (using positions), we can access the successor and predecessor of an element from a given position, we can access an element from a position.

- Every element from a list has a unique position in the list:
 - positions are relative to the list (but important for the list)
 - the position of an element:
 - identifies the element from the list
 - determines the position of the successor and predecessor element (if they exist).

- Position of an element can be seen in different ways:
 - as the *rank* of the element in the list (first, second, third, etc.)
 - similarly to an array, the position of an element is actually its index
 - as a *reference* to the memory location where the element is stored.
 - for example a pointer to the memory location
- For a general treatment, we will consider in the following the *position* of an element in an abstract manner, and we will consider that positions are of type *TPosition*

- A position p will be considered *valid* if it denotes the position of an actual element from the list:
 - if p is a pointer to a memory location, p is valid if it is the address of an element from a list (not NIL or some other address that is not the address of any element)
 - if p is the rank of the element from the list, p is valid if it is between 1 and the number of elements.
- For an invalid position we will use the following notation: $\perp$

- Domain of the ADT List:

$\mathcal{L} = \{l \mid l \text{ is a list with elements of type TElem, each having a unique position in } l \text{ of type TPosition}\}$

- **init(l)**
 - **descr:** creates a new, empty list
 - **pre:** true
 - **post:** $l \in \mathcal{L}$, l is an empty list

- **first(l)**
 - **descr:** returns the TPosition of the first element
 - **pre:** $l \in \mathcal{L}$
 - **post:** $first \leftarrow p \in TPosition$

$$p = \begin{cases} \text{the position of the first element from } l & \text{if } l \neq \emptyset \\ \perp & \text{otherwise} \end{cases}$$

- **last(l)**
 - **descr:** returns the TPosition of the last element
 - **pre:** $l \in \mathcal{L}$
 - **post:** $last \leftarrow p \in TPosition$
$$p = \begin{cases} \text{the position of the last element from } l & \text{if } l \neq \emptyset \\ \perp & \text{otherwise} \end{cases}$$

- $\text{valid}(l, p)$
 - **descr:** checks whether a TPosition is valid in a list
 - **pre:** $l \in \mathcal{L}, p \in TPosition$
 - **post:**

$$\text{valid} \leftarrow \begin{cases} \text{true} & \text{if } p \text{ is a valid position in } l \\ \text{false} & \text{otherwise} \end{cases}$$

- $\text{next}(l, p)$

- **descr:** goes to the next TPosition from a list
- **pre:** $l \in \mathcal{L}, p \in TPosition, \text{valid}(l, p)$
- **post:** $\text{next} \leftarrow q \in TPosition$

$$q = \begin{cases} \text{the pos. of the next element after } p & \text{if } p \text{ is not the last pos.} \\ \perp & \text{otherwise} \end{cases}$$

- **throws:** exception if p is not valid

- `previous(l, p)`

- **descr:** goes to the previous `TPosition` from a list
- **pre:** $l \in \mathcal{L}, p \in TPosition, \text{valid}(l, p)$
- **post:** $\text{previous} \leftarrow q \in TPosition$

$$q = \begin{cases} \text{the pos. of the element before } p & \text{if } p \text{ is not the first pos.} \\ \perp & \text{otherwise} \end{cases}$$

- **throws:** exception if p is not valid

- `getElement(l, p)`
 - **descr:** returns the element from a given `TPosition`
 - **pre:** $l \in \mathcal{L}, p \in TPosition, \text{valid}(l, p)$
 - **post:** $\text{getElement} \leftarrow e, e \in TElem, e = \text{the element from position } p \text{ from } l$
 - **throws:** exception if p is not valid

- $\text{position}(l, e)$
 - **descr:** returns the $TPosition$ of an element
 - **pre:** $l \in \mathcal{L}, e \in TElem$
 - **post:** $position \leftarrow p \in TPosition$

$$p = \begin{cases} \text{the first position of element } e \text{ from } l & \text{if } e \in l \\ \perp & \text{otherwise} \end{cases}$$

- **setElement**(l, p, e)
 - **descr:** replaces an element from a $TPosition$ with another
 - **pre:** $l \in \mathcal{L}, p \in TPosition, e \in TElem, \text{valid}(l, p)$
 - **post:** $l' \in \mathcal{L}$, the element from position p from l' is e ,
 $\text{setElement} \leftarrow el, el \in TElem, el$ is the element from position
 p from l (returns the previous value from the position)
 - **throws:** exception if p is not valid

- `addToBeginning(l, e)`
 - **descr:** adds a new element to the beginning of a list
 - **pre:** $l \in \mathcal{L}, e \in TElem$
 - **post:** $l' \in \mathcal{L}$, l' is the result after the element e was added at the beginning of l

- **addToEnd(l, e)**
 - **descr:** adds a new element to the end of a list
 - **pre:** $l \in \mathcal{L}, e \in TElem$
 - **post:** $l' \in \mathcal{L}$, l' is the result after the element e was added at the end of l

- **addBeforePosition**(l, p, e)
 - **descr:** inserts a new element before a given position (which means that the new element will be on that position)
 - **pre:** $l \in \mathcal{L}, p \in TPosition, e \in TElem, \text{valid}(l, p)$
 - **post:** $l' \in \mathcal{L}, l'$ is the result after the element e was added in l before the position p
 - **throws:** exception if p is not valid

- `addAfterPosition(l, p, e)`
 - **descr:** inserts a new element after a given position
 - **pre:** $l \in \mathcal{L}, p \in TPosition, e \in TElem, \text{valid}(l, p)$
 - **post:** $l' \in \mathcal{L}$, l' is the result after the element e was added in l after the position p
 - **throws:** exception if p is not valid

- `remove(l, p)`
 - **descr:** removes an element from a given position from a list
 - **pre:** $l \in \mathcal{L}, p \in TPosition, \text{valid}(l, p)$
 - **post:** $\text{remove} \leftarrow e, e \in TElem, e$ is the element from position p from $l, l' \in \mathcal{L}, l' = l - e$.
 - **throws:** exception if p is not valid

- **remove**(l, e)
 - **descr:** removes the first occurrence of a given element from a list
 - **pre:** $l \in \mathcal{L}, e \in TElem$
 - **post:**

$$remove \leftarrow \begin{cases} true & \text{if } e \in l \text{ and it was removed} \\ false & \text{otherwise} \end{cases}$$

- $\text{search}(l, e)$
 - **descr:** searches for an element in the list
 - **pre:** $l \in \mathcal{L}, e \in TElem$
 - **post:**

$$\text{search} \leftarrow \begin{cases} \text{true} & \text{if } e \in l \\ \text{false} & \text{otherwise} \end{cases}$$

- **isEmpty(l)**
 - **descr:** checks if a list is empty
 - **pre:** $l \in \mathcal{L}$
 - **post:**

$$isEmpty \leftarrow \begin{cases} true & \text{if } l = \emptyset \\ false & \text{otherwise} \end{cases}$$

- **size(l)**
 - **descr:** returns the number of elements from a list
 - **pre:** $l \in \mathcal{L}$
 - **post:** $size \leftarrow$ the number of elements from l

- `destroy(l)`
 - **descr:** destroys a list
 - **pre:** $l \in \mathcal{L}$
 - **post:** l was destroyed

- `iterator(l, it)`
 - **descr:** returns an iterator for a list
 - **pre:** $l \in \mathcal{L}$
 - **post:** $it \in \mathcal{I}$, it is an iterator over l , the current element from it is the first element from l , or, if l is empty, it is invalid

TPosition - Integer

- In Python and Java, TPosition is represented by an index.
- We can add and remove using index and we can access elements using their index (but we have iterator as well for the List).
- For example (Python):
insert (int index, E object)
index (E object)
 - Returns an integer value, position of the element (or exception if *object* is not in the list)
- For example (Java):
void add(int index, E element)
E get(int index)
E remove(int index)
 - Returns the removed element

- If we consider that TPosition is an Integer value (similar to Python and Java), we can have an *IndexedList*
- In case of an *IndexedList* the operations that work with a position take as parameter integer numbers representing these positions
- There are less operations in the interface of the *IndexedList*
 - Operations *first*, *last*, *next*, *previous*, *valid* do not exist

- **init(l)**
 - **descr:** creates a new, empty list
 - **pre:** true
 - **post:** $l \in \mathcal{L}$, l is an empty list

- `getElement(l, i)`
 - **descr:** returns the element from a given position
 - **pre:** $l \in \mathcal{L}, i \in \mathcal{N}, i$ is a valid position
 - **post:** $getElement \leftarrow e, e \in TElem, e =$ the element from position i from l
 - **throws:** exception if i is not valid

- $\text{position}(l, e)$
 - **descr:** returns the position of an element
 - **pre:** $l \in \mathcal{L}, e \in T\text{Elem}$
 - **post:** $\text{position} \leftarrow i \in \mathcal{N}$

$$i = \begin{cases} \text{the first position of element } e \text{ from } l & \text{if } e \in l \\ -1 & \text{otherwise} \end{cases}$$

- `setElement(l, i, e)`
 - **descr:** replaces an element from a position with another
 - **pre:** $l \in \mathcal{L}, i \in \mathcal{N}, e \in T\text{Elem}, i$ is a valid position
 - **post:** $l' \in \mathcal{L}$, the element from position i from l' is e ,
 $\text{setElement} \leftarrow el, el \in T\text{Elem}, el$ is the element from position i from l (returns the previous value from the position)
 - **throws:** exception if i is not valid

- **addToBeginning**(l, e)
 - **descr:** adds a new element to the beginning of a list
 - **pre:** $l \in \mathcal{L}, e \in TElem$
 - **post:** $l' \in \mathcal{L}$, l' is the result after the element e was added at the beginning of l

- **addToEnd(l, e)**
 - **descr:** adds a new element to the end of a list
 - **pre:** $l \in \mathcal{L}, e \in TElem$
 - **post:** $l' \in \mathcal{L}$, l' is the result after the element e was added at the end of l

- `addToPosition(l, i, e)`
 - **descr:** inserts a new element at a given position (it is the same as *addBeforePosition*)
 - **pre:** $l \in \mathcal{L}, i \in \mathcal{N}, e \in TElem$, i is a valid position (size + 1 is valid for adding an element)
 - **post:** $l' \in \mathcal{L}$, l' is the result after the element e was added in l at the position i
 - **throws:** exception if i is not valid

- `remove(l, i)`
 - **descr:** removes an element from a given position from a list
 - **pre:** $l \in \mathcal{L}, i \in \mathcal{N}$, i is a valid position
 - **post:** $remove \leftarrow e$, $e \in TElem$, e is the element from position i from l , $l' \in \mathcal{L}$, $l' = l - e$.
 - **throws:** exception if i is not valid

- `remove(l, e)`
 - **descr:** removes the first occurrence of a given element from a list
 - **pre:** $l \in \mathcal{L}, e \in TElem$
 - **post:**

$$remove \leftarrow \begin{cases} true & \text{if } e \in l \text{ and it was removed} \\ false & \text{otherwise} \end{cases}$$

- `search(l, e)`
 - **descr:** searches for an element in the list
 - **pre:** $l \in \mathcal{L}, e \in TElem$
 - **post:**

$$search \leftarrow \begin{cases} true & \text{if } e \in l \\ false & \text{otherwise} \end{cases}$$

- **isEmpty()**
 - **descr:** checks if a list is empty
 - **pre:** $l \in \mathcal{L}$
 - **post:**

$$isEmpty \leftarrow \begin{cases} true & \text{if } l = \emptyset \\ false & \text{otherwise} \end{cases}$$

- `size(l)`
 - **descr:** returns the number of elements from a list
 - **pre:** $l \in \mathcal{L}$
 - **post:** $size \leftarrow$ the number of elements from l

- **destroy(l)**
 - **descr:** destroys a list
 - **pre:** $l \in \mathcal{L}$
 - **post:** l was destroyed

- `iterator(l, it)`
 - **descr:** returns an iterator for a list
 - **pre:** $l \in \mathcal{L}$
 - **post:** $it \in \mathcal{I}$, it is an iterator over l , the current element from it is the first element from l , or, if l is empty, it is invalid

- In STL (C++), TPosition is represented by an iterator.

- For example - vector:

iterator insert(iterator position, const value_type& val)

- Returns an iterator which points to the newly inserted element

iterator erase (iterator position);

- Returns an iterator which points to the element after the removed one

- For example - list:

iterator insert(iterator position, const value_type& val)

iterator erase (iterator position);

- If we consider that TPosition is an Iterator (similar to C++) we can have an *IteratedList*.
- In case of an *IteratedList* the operations that take as parameter a position use an Iterator (and the position is the current element from the Iterator)
- Operations *valid*, *next*, *previous* no longer exist in the interface of the List (they are operations for the Iterator).

- **init(l)**
 - **descr:** creates a new, empty list
 - **pre:** true
 - **post:** $l \in \mathcal{L}$, l is an empty list

- $\text{first}(I)$
 - **descr:** returns an Iterator set to the first element
 - **pre:** $I \in \mathcal{L}$
 - **post:** $\text{first} \leftarrow it \in \text{Iterator}$

$$it \leftarrow \begin{cases} \text{an iterator set to the first element} & \text{if } I \neq \emptyset \\ \text{an invalid iterator} & \text{otherwise} \end{cases}$$

- last(l)

- **descr:** returns an Iterator set to the last element
- **pre:** $l \in \mathcal{L}$
- **post:** $last \leftarrow it \in Iterator$

$$it \leftarrow \begin{cases} \text{an iterator set to the last element} & \text{if } l \neq \emptyset \\ \text{an invalid iterator} & \text{otherwise} \end{cases}$$

- `getElement(l, it)`
 - **descr:** returns the element from the position denoted by an iterator
 - **pre:** $l \in \mathcal{L}, it \in \text{Iterator}, \text{valid}(it)$
 - **post:** $\text{getElement} \leftarrow e, e \in \text{TElem}, e = \text{the element from } l \text{ from the current position}$
 - **throws:** exception if it is not valid

- $\text{position}(l, e)$
 - **descr:** returns an iterator set to the first position of an element
 - **pre:** $l \in \mathcal{L}, e \in T\text{Elem}$
 - **post:** $\text{position} \leftarrow it \in \text{Iterator}$

$$it \leftarrow \begin{cases} \text{an iterator set to the first pos. of } e \text{ from } l & \text{if } e \in l \\ \text{an invalid iterator} & \text{otherwise} \end{cases}$$

- `setElement(l, it, e)`
 - **descr:** replaces the element from the position denoted by an iterator with another element
 - **pre:** $l \in \mathcal{L}, it \in \text{Iterator}, e \in \text{TElem}, \text{valid}(it)$
 - **post:** $l' \in \mathcal{L}$, the element from the position denoted by it from l' is e , $\text{setElement} \leftarrow el, el \in \text{TElem}, el$ is the element from the current position from it from l (returns the previous value from the position)
 - **throws:** exception if it is not valid

- **addToBeginning**(l, e)
 - **descr:** adds a new element to the beginning of a list
 - **pre:** $l \in \mathcal{L}, e \in TElem$
 - **post:** $l' \in \mathcal{L}$, l' is the result after the element e was added at the beginning of l

- **addToEnd(l, e)**
 - **descr:** inserts a new element at the end of a list
 - **pre:** $l \in \mathcal{L}, e \in TElem$
 - **post:** $l' \in \mathcal{L}, l'$ is the result after the element e was added at the end of l

- `addToPosition(l, it, e)`
 - **descr:** inserts a new element at a given position specified by the iterator (it is the same as *addAfterPosition*)
 - **pre:** $l \in \mathcal{L}, it \in \text{Iterator}, e \in \text{TElem}, \text{valid}(it)$
 - **post:** $l' \in \mathcal{L}, l'$ is the result after the element e was added in l at the position specified by it
 - **throws:** exception if it is not valid

- `remove(l, it)`
 - **descr:** removes an element from a given position specified by the iterator from a list
 - **pre:** $l \in \mathcal{L}, it \in \text{Iterator}, \text{valid}(it)$
 - **post:** $\text{remove} \leftarrow e, e \in \text{TElem}, e$ is the element from the position from l denoted by $it, l' \in \mathcal{L}, l' = l - e$.
 - **throws:** exception if it is not valid

- `remove(l, e)`
 - **descr:** removes the first occurrence of a given element from a list
 - **pre:** $l \in \mathcal{L}, e \in TElem$
 - **post:**

$$remove \leftarrow \begin{cases} true & \text{if } e \in l \text{ and it was removed} \\ false & \text{otherwise} \end{cases}$$

- `search(l, e)`
 - **descr:** searches for an element in the list
 - **pre:** $l \in \mathcal{L}, e \in TElem$
 - **post:**

$$search \leftarrow \begin{cases} true & \text{if } e \in l \\ false & \text{otherwise} \end{cases}$$

- **isEmpty()**
 - **descr:** checks if a list is empty
 - **pre:** $l \in \mathcal{L}$
 - **post:**

$$isEmpty \leftarrow \begin{cases} true & \text{if } l = \emptyset \\ false & \text{otherwise} \end{cases}$$

- `size(l)`
 - **descr:** returns the number of elements from a list
 - **pre:** $l \in \mathcal{L}$
 - **post:** $size \leftarrow$ the number of elements from l

- `destroy(l)`
 - **descr:** destroys a list
 - **pre:** $l \in \mathcal{L}$
 - **post:** l was destroyed

ADT SortedList I

- We can define the ADT *SortedList*, in which the elements are memorized in an order given by a relation.

- You have below the list of operations for ADT *List*

- `init(l)`
- `first(l)`
- `last(l)`
- `valid(l, p)`
- `next(l, p)`
- `previous(l, p)`
- `getElement(l, p)`
- `position(l, e)`
- `setElement(l, p, e)`
- `addToBeginning(l, e)`
- `addToEnd(l, e)`
- `addPosition(l, p, e)`
- `remove(l, p)`
- `remove(l, e)`
- `search(l, e)`
- `isEmpty(l)`
- `size(l)`
- `destroy(l)`
- `iterator(l, it)`

- Which operations do no longer exist for a *SortedList*? What operations should be added? Should we change the parameters of some operations?

- The interface of the ADT *SortedList* is very similar to that of the ADT *List* with some exceptions:
 - The *init* function takes as parameter a relation that is going to be used to order the elements
 - We no longer have several *add* operations (*addToBeginning*, *addToEnd*, *addToPostion*), we have one single *add* operation, which takes as parameter only the element to be added (and adds it to the position where it should go based on the relation)
 - We no longer have a *setElement* operation (might violate ordering)
- We can consider *TPosition* in two different ways for a *SortedList* as well $\Rightarrow$ *SortedListIndexed* and *SortedListIterated*

Dynamic Array - review

- The main idea of the (dynamic) array is that all the elements from the array are in one single consecutive memory location.

Dynamic Array - review

- The main idea of the (dynamic) array is that all the elements from the array are in one single consecutive memory location.
- This gives us the main advantage of the array:
 - constant time access to any element from any position
 - constant time for operations (add, remove) at the end of the array

Dynamic Array - review

- The main idea of the (dynamic) array is that all the elements from the array are in one single consecutive memory location.
- This gives us the main advantage of the array:
 - constant time access to any element from any position
 - constant time for operations (add, remove) at the end of the array
- This gives us the main disadvantage of the array as well:
 - $\Theta(n)$ complexity for operations (add, remove) at the beginning of the array

Linked Lists

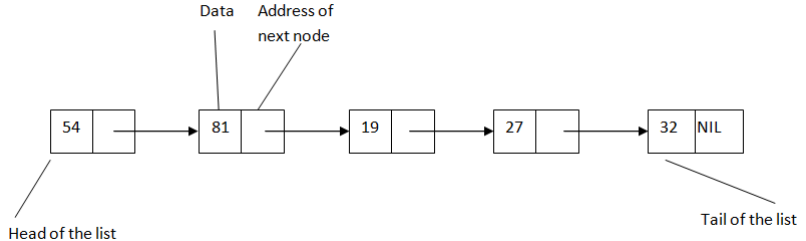
- A *linked list* is a linear data structure, where the order of the elements is determined not by indexes, but by a pointer which is placed in each element.
- A linked list is a structure that consists of *nodes* (sometimes called *links*) and each node contains, besides the data (that we store in the linked list), a pointer to the address of the next node (and possibly a pointer to the address of the previous node).
- The nodes of a linked list are not necessarily adjacent in the memory, this is why we need to keep the address of the successor in each node.

Linked Lists

- Elements from a linked list are accessed based on the pointers stored in the nodes.
- We can directly access only the first element (and maybe the last one) of the list.

Linked Lists

- Example of a linked list with 5 nodes:



Singly Linked Lists - SLL

- The linked list from the previous slide is actually a *singly linked list* - *SLL*.
- In a SLL each node from the list contains the data and the address of the next node.
- The first node of the list is called *head* of the list and the last node is called *tail* of the list.
- The tail of the list contains the special value *NIL* as the address of the next node (which does not exist).
- If the head of the SLL is *NIL*, the list is considered empty.

Singly Linked Lists - Representation

- For the representation of a SLL we need two structures: one structure for the node and one for the list itself.

SLLNode:

info: TElem *//the actual information*

next: ↑ SLLNode *//address of the next node*

Singly Linked Lists - Representation

- For the representation of a SLL we need two structures: one structure for the node and one for the list itself.

SLLNode:

info: TElem *//the actual information*

next: ↑ SLLNode *//address of the next node*

SLL:

head: ↑ SLLNode *//address of the first node*

- Usually, for a SLL, we only memorize the address of the head. However, there might be situations when we memorize the address of the tail as well (if it helps us implement the operations).

- Possible operations for a singly linked list:
 - search for an element with a given value
 - add an element (to the beginning, to the end, to a given position, after a given value)
 - delete an element (from the beginning, from the end, from a given position, with a given value)
 - get an element from a position
- These are *possible* operations; usually we need only part of them, depending on the container that we implement using a SLL.

function search (sll, elem) **is:**

//pre: sll is a SLL - singly linked list; elem is a TElem

//post: returns the node which contains elem as info, or NIL

function search (sll, elem) **is:**

//pre: sll is a SLL - singly linked list; elem is a TElem

//post: returns the node which contains elem as info, or NIL

current $\leftarrow$ sll.head

while current $\neq$ NIL **and** [current].info $\neq$ elem **execute**

current $\leftarrow$ [current].next

end-while

search $\leftarrow$ current

end-function

- Complexity:

function search (sll, elem) **is:**

//pre: sll is a SLL - singly linked list; elem is a TElem

//post: returns the node which contains elem as info, or NIL

current $\leftarrow$ sll.head

while current $\neq$ NIL **and** [current].info $\neq$ elem **execute**

current $\leftarrow$ [current].next

end-while

search $\leftarrow$ current

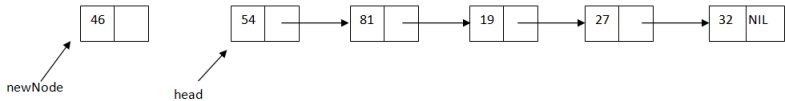
end-function

- Complexity: $O(n)$ - we can find the element in the first node, or we may need to verify every node.
- **Obs:** this implementation returns a node. When we implement a container, search is not allowed to return a node (we hide the representation), we return a boolean flag or a position. This implementation might be a private helper function for other functions.

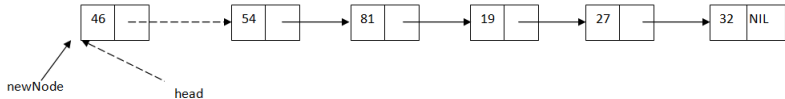
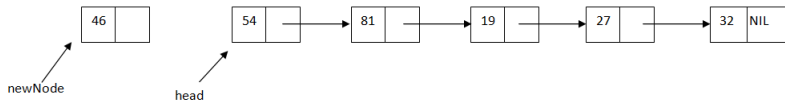
SLL - Walking through a linked list

- In the *search* function we have seen how we can walk through the elements of a linked list:
 - we need an auxiliary node (called *current*), which starts at the head of the list
 - at each step, the value of the *current* node becomes the address of the successor node (through the $current \leftarrow [current].next$ instruction)
 - we stop when the current node becomes *NIL*

SLL - Insert at the beginning



SLL - Insert at the beginning



SLL - Insert at the beginning

subalgorithm insertFirst (sll, elem) **is:**

//pre: sll is a SLL; elem is a TElem

//post: the element elem will be inserted at the beginning of sll

newNode $\leftarrow$ allocate() *//allocate a new SLLNode*

[newNode].info $\leftarrow$ elem

[newNode].next $\leftarrow$ sll.head

sll.head $\leftarrow$ newNode

end-subalgorithm

- Complexity:

SLL - Insert at the beginning

subalgorithm insertFirst (sll, elem) **is:**

//pre: sll is a SLL; elem is a TElem

//post: the element elem will be inserted at the beginning of sll

newNode $\leftarrow$ allocate() *//allocate a new SLLNode*

[newNode].info $\leftarrow$ elem

[newNode].next $\leftarrow$ sll.head

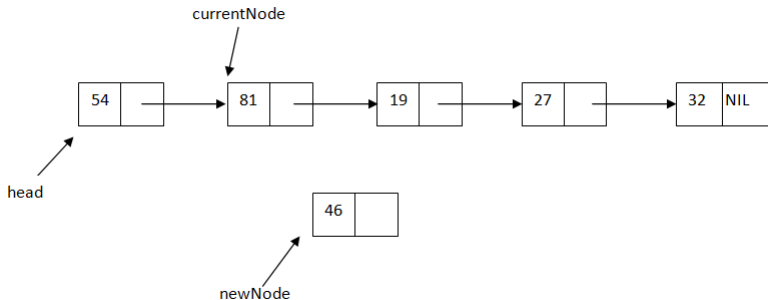
sll.head $\leftarrow$ newNode

end-subalgorithm

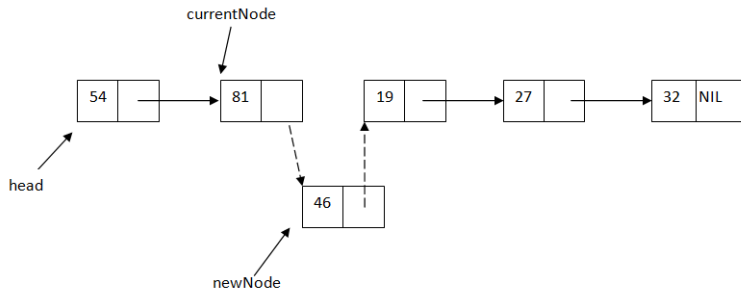
- Complexity: $\Theta(1)$

SLL - Insert after a node

- Suppose that we have the address of a node from the SLL (maybe because the search operation returned it) and we want to insert a new element after that node.



SLL - Insert after a node



SLL - Insert after a node

subalgorithm insertAfter(sll, currentNode, elem) **is:**

//pre: sll is a SLL; currentNode is an SLLNode from sll;

//elem is a TElem

//post: a node with elem will be inserted after node currentNode

newNode $\leftarrow$ allocate() *//allocate a new SLLNode*

[newNode].info $\leftarrow$ elem

[newNode].next $\leftarrow$ [currentNode].next

[currentNode].next $\leftarrow$ newNode

end-subalgorithm

- Complexity:

SLL - Insert after a node

subalgorithm insertAfter(sll, currentNode, elem) **is:**

//pre: sll is a SLL; currentNode is an SLLNode from sll;

//elem is a TElem

//post: a node with elem will be inserted after node currentNode

newNode $\leftarrow$ allocate() *//allocate a new SLLNode*

[newNode].info $\leftarrow$ elem

[newNode].next $\leftarrow$ [currentNode].next

[currentNode].next $\leftarrow$ newNode

end-subalgorithm

- Complexity: $\Theta(1)$

Insert before a node

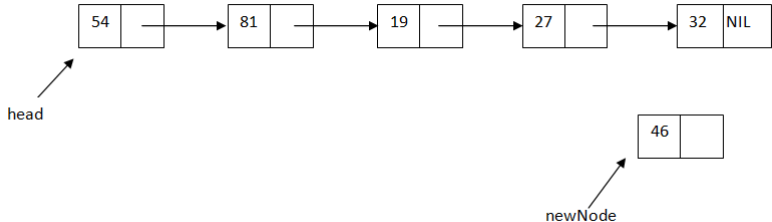
- Think about the following case: if you have a node, how can you insert an element in front of the node?

SLL - Insert at a position

- We usually do not have the node after which we want to insert an element: we either know the position to which we want to insert, or know the element (not the node) after which we want to insert an element.
- Suppose we want to insert a new element at integer position p (after insertion the new element will be at position p). Since we only have access to the *head* of the list we first need to find the position *after* which we insert the element.

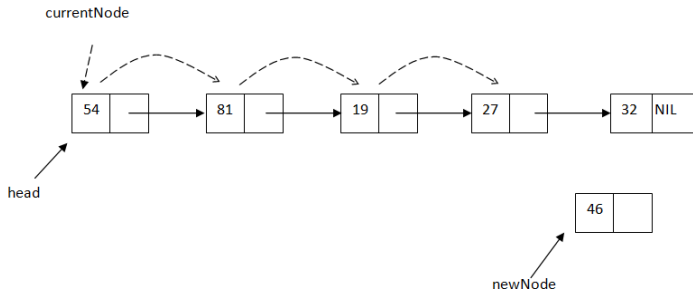
SLL - Insert at a position

- We want to insert element 46 at position 5.



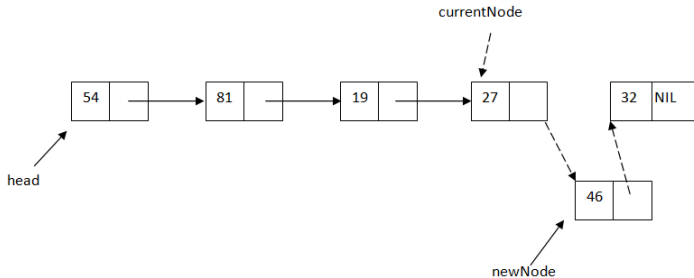
SLL - Insert at a position

- We need the 4th node (to insert element 46 after it), but we have direct access only to the first one, so we have to take an auxiliary node (*currentNode*) to get to the position.



SLL - Insert at a position

- Now we insert after node *currentNode*



SLL - Insert at a position

subalgorithm insertPosition(sll, pos, elem) **is:**

//pre: sll is a SLL; pos is an integer number; elem is a TElem

//post: a node with TElem will be inserted at position pos

if pos < 1 **then**

 @error, invalid position

else if pos = 1 **then** *//we want to insert at the beginning*

 newNode ← allocate() *//allocate a new SLLNode*

 [newNode].info ← elem

 [newNode].next ← sll.head

 sll.head ← newNode

else

 currentNode ← sll.head

 currentPos ← 1

while currentPos < pos - 1 **and** currentNode ≠ NIL **execute**

 currentNode ← [currentNode].next

 currentPos ← currentPos + 1

end-while

//continued on the next slide...

```
if currentNode  $\neq$  NIL then
    newNode  $\leftarrow$  allocate() //allocate a new SLLNode
    [newNode].info  $\leftarrow$  elem
    [newNode].next  $\leftarrow$  [currentNode].next
    [currentNode].next  $\leftarrow$  newNode
else
    @error, invalid position
end-if
end-if
end-subalgorithm
```

- Complexity:

```
if currentNode  $\neq$  NIL then
    newNode  $\leftarrow$  allocate() //allocate a new SLLNode
    [newNode].info  $\leftarrow$  elem
    [newNode].next  $\leftarrow$  [currentNode].next
    [currentNode].next  $\leftarrow$  newNode
else
    @error, invalid position
end-if
end-if
end-subalgorithm
```

- Complexity: $O(n)$

Get element from a given position

- Since we only have access to the head of the list, if we want to get an element from a position p we have to go through the list, node-by-node until we get to the p^{th} node.
- The process is similar to the first part of the *insertPosition* subalgorithm

SLL - Delete a given element

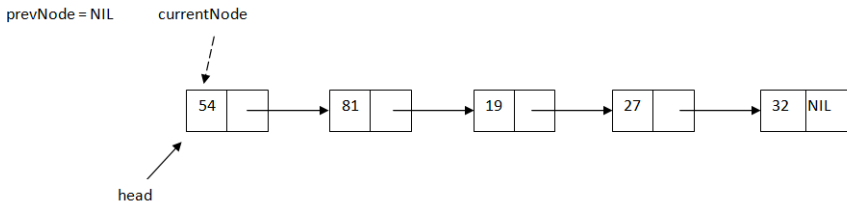
- How do we delete a given element from a SLL?

SLL - Delete a given element

- How do we delete a given element from a SLL?
- When we want to delete a node from the middle of the list (either a node with a given element, or a node from a position), we need to find the node *before* the one we want to delete.
- The simplest way to do this, is to walk through the list using two pointers: *currentNode* and *prevNode* (the node before *currentNode*). We will stop when *currentNode* points to the node we want to delete.

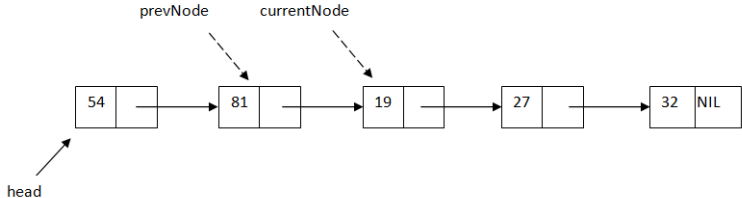
SLL - Delete a given element

- Suppose we want to delete the node with information 19.



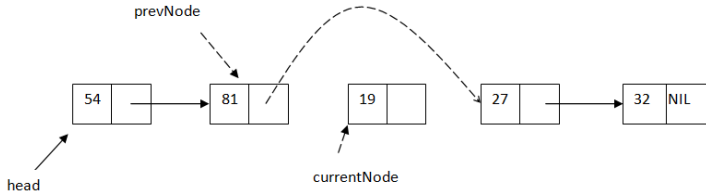
SLL - Delete a given element

- Move with the two pointers until *currentNode* is the node we want to delete.



SLL - Delete a given element

- Delete *currentNode* by *jumping over it*



SLL - Delete a given element

function deleteElement(sll, elem) **is:**

//pre: sll is a SLL, elem is a TElem

//post: the node with elem is removed from sll and returned

currentNode $\leftarrow$ sll.head

prevNode $\leftarrow$ NIL

while currentNode $\neq$ NIL **and** [currentNode].info $\neq$ elem **execute**

prevNode $\leftarrow$ currentNode

currentNode $\leftarrow$ [currentNode].next

end-while

if currentNode $\neq$ NIL **AND** prevNode = NIL **then** *//we delete the head*

sll.head $\leftarrow$ [sll.head].next

else if currentNode $\neq$ NIL **then**

[prevNode].next $\leftarrow$ [currentNode].next

[currentNode].next $\leftarrow$ NIL

end-if

deleteElement $\leftarrow$ currentNode

end-function

SLL - Delete a given element

- Complexity of *deleteElement* function:

SLL - Delete a given element

- Complexity of *deleteElement* function: $O(n)$

- Today we have talked about:
 - ADT Queue
 - ADT PriorityQueue
 - ADT Deque
 - ADT List and ADT SortedList
- Singly Linked lists